# Komikan Concurrente

## Informe técnico de sincronización y evaluación

### 1. Arquitectura y recursos compartidos

Un proceso simula N clientes locales mediante hilos. Cada cliente representa un bando; no es una pieza. Un emparejador consume una cola FIFO de llegadas y reserva hasta M mesas. Cada mesa activa tiene un juez independiente y trece hilos de piezas, con retardos locales aleatorios entre 50 y 300 ms. Los clientes y el emparejador esperan mediante variables de condición, sin espera activa.

### 2. Exclusión mutua y validación del juez

El mutex del torneo protege la cola, asignaciones, contadores y estados de slots. Cada mesa tiene otro mutex que protege su tablero, estado terminal y buzones de intenciones. Las piezas calculan un destino desde una vista consistente y publican una intención con secuencia. El juez consume las solicitudes en ese orden, revalida adyacencia, ocupación y captura contra el estado vigente y aplica la jugada en una única sección crítica. Si dos intenciones reclaman una casilla, solo la primera todavía válida puede ocuparla.

### 3. Reglas e invariantes

El tablero es un grafo explícito de 28 nodos: 25 de cuadrícula y tres del trono. Se siguen las líneas y círculos de la figura; no se añade una posición en el cruce sin círculo ni diagonales en la cuadrícula. Un salto recorre dos aristas colineales hacia adelante. Se acepta longitud gráfica desigual en la columna central del trono, una ambigüedad documentada. El tablero conserva una posición por pieza viva, una pieza por nodo y perros vivos + capturados = 12. El Puma gana con seis capturas; los Perros ganan si no hay paso ni salto legal.

### 4. Finalización y propiedad de los hilos

Las primitivas de mesa se inicializan antes de admitir clientes y se conservan durante la reutilización del slot. El juez une todas sus piezas antes de avisar al torneo; el emparejador une al juez antes de reasignar la mesa. Las notificaciones tardías de abandono verifican el ID del cliente. SIGINT se bloquea antes de crear hilos y lo consume sigwait en un hilo dedicado. El cierre detiene nuevas admisiones, despierta esperas, cancela administrativamente partidas y une los hilos antes de destruir recursos. SIGUSR1 despierta al hilo de señales al finalizar normalmente; no se usa pthread_cancel.

# Riesgos y evaluación

### 5. Deadlock, livelock y starvation

Los mutexes de torneo, tablero y salida no se mantienen anidados. El registro de eventos se hace tras copiar el tablero y liberar su mutex. Las esperas temporizadas usan CLOCK_MONOTONIC y liberan el mutex; los despertares espurios preservan el plazo original. Todos los predicados se revisan en bucles. El juez no espera un turno ni a una pieza concreta: consume cualquier intención pendiente. Esto evita ciclos de espera y una ficha inmóvil no detiene al resto.

Los reintentos se espacian con retardos aleatorios; no hay bucles de sondeo continuo. La cola FIFO evita adelantar clientes ya admitidos y las intenciones se atienden por secuencia. POSIX no promete equidad estricta al adquirir mutexes ni planificación justa: no se puede demostrar ausencia absoluta de starvation. Tampoco se garantiza que el movimiento aleatorio alcance una victoria en tiempo finito. El PDF no define empate ni límite temporal; SIGINT permite cerrar esas ejecuciones sin inventar ganadores.

### 6. Fallos y abandono

El cliente puede notificar un abandono cooperativo, que declara ganador al oponente. Se comprueban reservas, inicialización de primitivas y creación de hilos. Un fallo parcial de piezas despierta y une las ya creadas; un fallo de infraestructura detiene el torneo. No se intenta destruir un mutex retenido por un hilo arbitrariamente cancelado: la solución evita esa situación mediante salida cooperativa. No recupera corrupción arbitraria de memoria, SIGKILL ni la caída del proceso completo.

### 7. Complejidad, rendimiento y concurrencia

Se usan N hilos de clientes, hasta 14 por mesa activa y tres contextos de control (principal, emparejador, señales). El estado ocupa O(N + M), con tablero y buzones de tamaño constante. Un mutex por tablero simplifica la atomicidad y evita bloqueos entre celdas. El juez serializa las modificaciones de una misma partida, pero las decisiones de piezas y las mesas progresan de forma concurrente. El log completo facilita auditoría y puede limitar el rendimiento; no se presenta un benchmark de throughput ni una cifra de latencia no medida.

### 8. Evidencia y límites de verificación

La compilación real usa gcc -Wall -Wextra -std=c17 y -pthread, sin advertencias. Las pruebas cubren reglas, capturas, invariantes, contención, FIFO, configuración inválida, reutilización, 100 clientes, hasta cuatro mesas simultáneas, abandono y SIGINT. Se inyectan diez fallos de pthread_create. ASan/UBSan y TSan se ejecutan sobre las suites compatibles; los comandos y resultados exactos están en docs/pruebas.md y docs/evidencia/. LeakSanitizer no puede verificar fugas en este entorno: su runtime informa incompatibilidad con ptrace. No se certifica por ello cero fugas. GitHub Actions también pasó test y sanitizers en la ejecución 41734201956, incluido el escenario normal de 100 clientes con LeakSanitizer activo. Esta evidencia verifica ese escenario remoto; no certifica todos los cierres posibles.